

# Complete Continuous Line-by-Line Code Analysis: agent/outreach.py

Exhaustive walk-through of all 523 lines — B2B Lead Scraper & Cold Email Engine (Zero Page Breaks / Gaps)

## PART 1: MODULE DOCSTRING & CORE IMPORTS (LINES 1–12)

| Lines | Source Code Snippet                                                   | Detailed Explanation                                                    |
|-------|-----------------------------------------------------------------------|-------------------------------------------------------------------------|
| 1     | <code>"""</code>                                                      | Opening triple-quote for the module-level docstring.                    |
| 2     | <code>agent/outreach.py</code>                                        | Specifies module path within the project architecture.                  |
| 3     | <code>B2B Lead Scraper + Cold Email Engine.</code>                    | High-level description of module functionalities.                       |
| 4     | <code>"""</code>                                                      | Closing triple-quote for module docstring.                              |
| 5     | <code></code>                                                         | Blank line separating docstring from import section.                    |
| 6     | <code>import json</code>                                              | Standard JSON parser for persisting and reading lead records.           |
| 7     | <code>import os</code>                                                | Operating system module for filesystem operations and path resolutions. |
| 8     | <code>import re</code>                                                | Regular expressions for string normalization and slug generation.       |
| 9     | <code>import time</code>                                              | Time utilities for rate-limiting email dispatches via delays.           |
| 10    | <code>from typing import Callable, Dict, List, Optional, Tuple</code> | Imports type annotations for enhanced static code inspection.           |
| 11    | <code>from urllib.parse import urlparse</code>                        | Parses domain hostnames from web URLs.                                  |
| 12    | <code></code>                                                         | Blank line separator.                                                   |

## PART 2: DIRECTORY PATHS & GLOBAL CONSTANTS (LINES 13–44)

| Lines | Source Code Snippet                                                                                       | Detailed Explanation                                                                                            |
|-------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 13    | <code>BASE_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))</code>                    | Calculates absolute path of root project directory relative to file location.                                   |
| 14    | <code>DATA_DIR = os.path.join(BASE_DIR, "data")</code>                                                    | Resolves the target `data/` storage directory path.                                                             |
| 15    | <code>LEADS_FILE = os.path.join(DATA_DIR, "leads.json")</code>                                            | Defines path to `leads.json` database file.                                                                     |
| 16    | <code></code>                                                                                             | Blank line separator.                                                                                           |
| 17    | <code>SENDER_EMAIL = "yeahboyadi@gmail.com"</code>                                                        | Default Gmail address used as sender in outreach campaigns.                                                     |
| 18    | <code>PLATFORM_URL = "https://skilluphackathon2026..."</code>                                             | Live web application URL included in call-to-action emails.                                                     |
| 19    | <code></code>                                                                                             | Blank line separator.                                                                                           |
| 20    | <code>COLD_EMAIL_SUBJECT = "25 Free AI Profiling Credits for {agency_name} – Detective Agentic AI"</code> | Email subject line template containing the `{agency_name}` variable.                                            |
| 21    | <code></code>                                                                                             | Blank line separator.                                                                                           |
| 22–44 | <code>COLD_EMAIL_BODY = """Hello {agency_name}, ..."""</code>                                             | Multi-line string template for email body with `{agency_name}`, `{location}`, and `{platform_url}` format tags. |

## PART 3: LEAD FILE IO & HELPER UTILITIES (LINES 45–94)

| Lines | Source Code Snippet                                                                               | Detailed Explanation                                                |
|-------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 45–47 | <code>def _ensure_data_dir() -&gt; None:<br/><br/>    os.makedirs(DATA_DIR, exist_ok=True)</code> | Ensures `data/` directory exists without throwing error if present. |

| Lines | Source Code Snippet                                                                                                                                                                                          | Detailed Explanation                                                                                                                                      |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 48–58 | <pre>def load_leads() -&gt; List[Dict]:      _ensure_data_dir()      if not os.path.exists(LEADS_FILE): re turn []      try: ... catch: return []</pre>                                                      | Loads and validates lead dictionary list from `leads.json`. Catches corruption or missing file errors cleanly.                                            |
| 59–63 | <pre>def save_leads(leads: List[Dict]) -&gt; None:      _ensure_data_dir()      with open(LEADS_FILE, "w", encoding ="utf-8") as file:          json.dump(leads, file, indent=2, ensure_ascii=False)</pre>   | Persists updated lead dictionary list back to `leads.json` with 2-space formatting.                                                                       |
| 64–68 | <pre>_HEADERS = {      "User-Agent": "DetectiveAgenticAI/1.0 (B2B lead discovery application)",      "Accept": "application/json, text/htm l, ...",      "Accept-Language": "en-IN,en;q=0.9",  }</pre>       | HTTP Request headers dictionary to identify client requests during scraping.                                                                              |
| 69–72 | <pre>def _get(url: str, params: Optional[dict] = None, timeout: int = 15):      import requests      return requests.get(url, params=params, headers=_HEADERS, timeout=timeout, allow_redire cts=True)</pre> | Internal wrapper function for executing GET requests via `requests` library.                                                                              |
| 73–94 | <pre>_slug(text)      _normalise_key(value)      _safe_email(value)      _domain_from_url(url)      _generated_email(name)</pre>                                                                             | Sanitization utilities for lowercase alphanumeric slugs, key normalization, email cleaning, domain extraction, and fallback emails across lines 73 to 94. |

## PART 4: HEURISTIC PROFESSION DETECTION (LINES 95–130)

| Lines   | Source Code Snippet                                                                                                                                                                                                                                                                             | Detailed Explanation                                                                  |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 95–104  | <pre>def _detect_profession(name: str, source: str, we bsite: str) -&gt; str:      """Heuristic to detect target agency profession..."""      try:          s, src, w = (name or "").lower(), (source or "").lower(), (website or "").lower()          except Exception: return "Unknown"</pre> | Extracts and converts input strings to lowercase for multi-category keyword matching. |
| 105–108 | <pre>if "police" in s or "police" in src or "amenity" in src or ".gov" in w or "gov.in" in w:      return "Police"</pre>                                                                                                                                                                        | Detects law enforcement agencies based on official `.gov` domains or keywords.        |

| Lines   | Source Code Snippet                                                                                                                                                                      | Detailed Explanation                                               |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| 109–115 | <pre> lawyer_terms = ["law", "advocat", "attorney", "so licitor", "counsel", "lawyer", "legal"]  for t in lawyer_terms:      if t in s or t in src or t in w: retu rn "Lawyer" </pre>    | Identifies legal practices and attorneys from keyword matches.     |
| 116–122 | <pre> detective_terms = ["detective", "investigat", "in quiry", "surveillance", ...]  for t in detective_terms:      if t in s or t in src or t in w: retu rn "Private Detective" </pre> | Categorizes private investigative agencies and surveillance firms. |
| 123–126 | <pre> if "security" in s or "security" in src or "secur ity" in w:      return "Security" </pre>                                                                                         | Detects commercial security firms and guard services.              |
| 127–130 | <pre> return "Unknown" </pre>                                                                                                                                                            | Default fallback status when no matching keywords are identified.  |

## PART 5: GEOCODING & OPENSTREETMAP OVERPASS SEARCH (LINES 131–210)

| Lines   | Source Code Snippet                                                                                                                                                                                                                               | Detailed Explanation                                                                                                   |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 131–148 | <pre> _OSM_TAG_MAP = { ... }  def _osm_tags_for_keyword(keyword: str) - &gt; List[Tuple[str, str]] </pre>                                                                                                                                         | Maps user search terms to key-value tags in OpenStreetMap database schemas.                                            |
| 149–162 | <pre> def _geocode_location(location: str) -&gt; Tuple[Opt ional[float], Optional[float], str]:      response = _get("https://nominatim.op enstreetmap.org/search", ...)      return float(item["lat"]), float(item ["lon"]), display_name </pre> | Resolves city or region string inputs into exact latitude/longitude GPS coordinates via Nominatim.                     |
| 163–182 | <pre> def _overpass_search(keyword: str, location: str, max_results: int) -&gt; List[Dict]:      lat, lon, _ = _geocode_location(locat ion)      tags = _osm_tags_for_keyword(keyword)      overpass_query = "[out:json][timeout: 25];..." </pre> | Constructs Overpass QL queries searching a 25,000m radius for node/way elements matching tags.                         |
| 183–192 | <pre> response = _get("https://overpass-api.de/api/inte rpreter", params={"data": overpass_query})  data = response.json() </pre>                                                                                                                 | Dispatches query payload to Overpass API servers with 30-second execution timeout.                                     |
| 193–210 | <pre> for element in data.get("elements", []):      tags = element.get("tags", {})      leads.append({ "agency_name": name, "contact_email": email, "source": "OpenStreetMap", ... }) </pre>                                                      | Parses response XML/JSON elements into standardized lead dictionaries, extracting websites, phone numbers, and emails. |

## PART 6: WEB SEARCH ENGINES & SYNTHETIC SEED GENERATION (LINES 211–300)

| Lines   | Source Code Snippet                                                                                                                                                                                                                                | Detailed Explanation                                                                                      |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 211–250 | <pre>def _duckduckgo_search(keyword: str, location: str, max_results: int) -&gt; List[Dict]:      query = f"{keyword} agencies {location}"      response = _get("https://api.duckduckgo.com/", ...)      ... extract related topics and URLs</pre> | Queries DuckDuckGo Instant Answer API to gather agency listings from web results across lines 211 to 250. |
| 251–277 | <pre>def _wikipedia_search(keyword: str, location: str, max_results: int) -&gt; List[Dict]:      query = f"{keyword} {location}"      response = _get("https://en.wikipedia.org/w/api.php", params={"action": "opensearch", ...})</pre>            | Searches English Wikipedia articles for recognized entities matching investigative or legal keywords.     |
| 278–300 | <pre>_AGENCY_SUFFIXES = [ "Detective Agency", "Investigation Services", ... ]  def _seed_leads(keyword, location, max_results):      ... create location-based fallback agency names</pre>                                                         | Generates placeholder leads with synthetic business suffixes for manual verification in remote locations. |

## PART 7: LEAD SCRAPING ORCHESTRATOR (LINES 301–377)

| Lines   | Source Code Snippet                                                                                                                                                                                                                                                                                                                       | Detailed Explanation                                                                                           |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 301–325 | <pre>def scrape_leads_sync(keyword: str, location: str, max_results: int = 20, allow_generated: bool = False) -&gt; List[Dict]:      max_results = max(1, min(int(max_results), 100))      for engine in (_overpass_search, _duckduckgo_search, _wikipedia_search):          collected.extend(engine(keyword, location, remaining))</pre> | Main entry function for lead scraping. Sequentially executes enabled search engines up to target yield limits. |
| 326–335 | <pre>if allow_generated and len(collected) &lt; max_results:      collected.extend(_seed_leads(keyword, location, remaining))</pre>                                                                                                                                                                                                       | Appends fallback synthetic seed leads only when explicitly permitted by `allow_generated=True`.                |
| 336–355 | <pre>for lead in collected:      confidence = 90 if "openstreetmap" in src else (70 if src in ("duckduckgo", "wikipedia") else 10)      profession = _detect_profession(...)</pre>                                                                                                                                                        | Calculates data confidence score (10–90%) and runs profession categorization on every discovered lead.         |
| 356–377 | <pre>existing = load_leads()      ... merge unique new leads into existing_keys      save_leads(merged)      return added if added else unique</pre>                                                                                                                                                                                      | Deduplicates gathered leads against current storage, merges new entries, saves to disk, and returns results.   |

## PART 8: COLD EMAIL INITIALIZATION & SETUP (LINES 378–410)

| Lines   | Source Code Snippet                                                                                                                                                                                            | Detailed Explanation                                                                                    |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 378–385 | <pre>def send_cold_emails(leads: List[Dict], app_password: str, ...):      if not leads: return []      if not app_password: return [{"status": "INIT_FAIL", "error": "Gmail App Password is missing."}]</pre> | Validates execution prerequisites: verifies non-empty input lead list and mandatory Gmail App Password. |
| 386–394 | <pre>try: import yagmail      except ImportError: return [{"status": "INIT_FAIL", "error": "yagmail is not installed."}]      yag = yagmail.SMTP(SENDER_EMAIL, str(app_password).strip())</pre>                | Dynamically imports `yagmail` and establishes an authenticated SMTP connection session with Gmail.      |
| 395–410 | <pre>total = len(leads)      for index, lead in enumerate(leads, start=1):          agency = lead.get("agency_name") or "Agency"          recipient = _safe_email(lead.get("contact_email"))</pre>             | Iterates through input targets, extracting business names and recipient email addresses.                |

## PART 9: EMAIL TEMPLATE FORMATTING & SENDING (LINES 411–480)

| Lines   | Source Code Snippet                                                                                                                                                                                                     | Detailed Explanation                                                                                       |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| 411–420 | <pre>if not recipient or "@" not in recipient:      results.append({"agency_name": agency, "status": "SKIPPED", "error": "No valid email address."})</pre>                                                              | Validates recipient email address syntax. Skips dispatch if invalid without failing the pipeline.          |
| 421–435 | <pre>format_values = { "agency_name": agency, "location": location, "platform_url": PLATFORM_URL, ... }      subject = subject_template.format(**format_values)      body = body_template.format(**format_values)</pre> | Populates subject and body templates with dynamically generated lead metadata variables.                   |
| 436–445 | <pre>yag.send(to=recipient, subject=subject, contents=body)      results.append({"agency_name": agency, "recipient": recipient, "status": "SENT", "error": ""})</pre>                                                   | Transmits email via Gmail SMTP and logs a successful `SENT` status record.                                 |
| 446–455 | <pre>except Exception as exc:      results.append({"agency_name": agency, "recipient": recipient, "status": "FAILED", "error": str(exc)})</pre>                                                                         | Catches SMTP delivery errors gracefully and logs a `FAILED` result entry.                                  |
| 456–480 | <pre>if progress_callback: progress_callback(index, total, agency, status, error)      if index &lt; total and delay_seconds &gt; 0: time.sleep(delay_seconds)</pre>                                                    | Triggers progress callback for Streamlit UI updates and imposes anti-spam delay pauses between dispatches. |

## PART 10: CLEANUP & PUBLIC EXPORTS (LINES 481–523)

| Lines   | Source Code Snippet                                                                                                                                                   | Detailed Explanation                                                                                       |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| 481-505 | <pre>finally:      try: yag.close()      except Exception: pass      return results</pre>                                                                             | Guarantees SMTP connection cleanup in a `finally` block and returns structured dispatch results list.      |
| 506-523 | <pre>__all__ = [      "scrape_leads_sync",      "send_cold_emails",      "load_leads",      "save_leads",      "COLD_EMAIL_SUBJECT",      "COLD_EMAIL_BODY",  ]</pre> | Explicitly defines the public API interface exposed when importing `agent/outreach.py` (lines 506 to 523). |